

問 3

店舗テーブル (shop) に登録されていない店舗に関する情報を、月間売り上げテーブル (sales) から削除してください。

問 4

貸し出し記録テーブル (rental) 上、returned列が9 (紛失) であるレコードについて、対応する書籍情報テーブル (books) 上の書籍情報を削除してみましょう。

chapter

4

データベース構造を 操作しよう

これまで SQL 命令の中でもデータそれ自体を操作するための「データ操作言語」について紹介してきました。アプリケーションを構築する中で、使用する 9 割がたはデータ操作命令ですが、データベースを理解するうえで、「データ定義言語」の理解は欠かせません。

データ定義言語については、データ型や細かな定義部分でデータベース製品ごとの差異が見られますが、本書ではごく基本的な部分と概念についてのみ学んでいくことにします。データベースごとの仕様については、各データベース製品のリファレンスやその他、専門のドキュメントを参照するようにしてください。

※ 注) なお、本章の例題や練習問題を実際に動作する場合には、新規のデータベースを作成して行うことをお勧めします。テーブルやインデックス定義が重複している場合、SQL 命令はエラーを返します。

4-1 テーブルを作成したい CREATE TABLE 命令

MySQL PostgreSQL SQLite SQL Server Oracle



Case リレーショナルデータベースにおいて、データを管理する基本的な単位は「テーブル」です。効率的なデータの分析、整理を行うために、最初によく練りこまれたテーブルを作成しておくことは重要です。

Q アンケート回答テーブル (quest) を作成してみましょう。quest テーブルのフィールド・レイアウトは P.35 の通りです。

A 新規にテーブルを作成するのは、CREATE TABLE 命令の役割です。

```
> CREATE TABLE
>   quest
>   (
>     id INT AUTO_INCREMENT,
>     name VARCHAR(100) NOT NULL,
>     name_kana VARCHAR(255) NOT NULL,
>     sex VARCHAR(5) NOT NULL,
>     prefecture VARCHAR(10) NOT NULL,
>     age INT DEFAULT 0,
>     answer1 INT NULL,
>     answer2 TEXT NULL,
>     answered DATETIME NOT NULL,
>     PRIMARY KEY (id)
>   )
> ;
Query OK, 0 rows affected (0.09 sec)
```

アンケート回答テーブル (quest)				
id	name	sex	...	answered
整数で 連番 (主キー)	文字列で 100 桁以内 NULL は 不可	文字列で 5 桁以内 NULL は 不可	...	日付時刻

データを格納する箱を新たに作成

構文を理解しよう

テーブルを定義する CREATE TABLE 命令の構文は以下の通りです。CREATE TABLE 命令にはいくつかの構文がありますが、一般的に利用可能な構文をここでは紹介します。

構文 CREATE TABLE 命令

CREATE TABLE テーブル名 (列名 データ型 列フラグ, ..., オプション);

テーブルを作成しなさい

名前は?

含まれる列は?

構文を日本語的に表現するならば、「<テーブル名>を指定された<列名 ...>の構成で作成しなさい」というわけです。利用可能なデータ型については P.10 で紹介した通り、データベース製品によってかなり異なりますので、詳細は各製品のドキュメント、及び、配布サンプルの .sql ファイルを参考にしてください。

列フラグとは、各列の特性を表すための属性情報で、以下のようなキーワードを指定することができます。列フラグは、各列ごとにデータ型の後に半角空白区切りで指定します (複数のフラグを指定することもできます)。

●主な列フラグ

列フラグ	概要
AUTO_INCREMENT	自動連番 (データ型は整数型)
DEFAULT 'デフォルト値'	値が未指定の場合のデフォルト値
[NOT] NULL	NULL を許容する (しない)
PRIMARY KEY	重複する値を許さない (NULL 値は不可)
UNIQUE	重複する値を許さない (NULL 値を許可)
REFERENCES テーブル名 (列名, ...)	外部参照制約

PRIMARY KEY (主キー) と **UNIQUE (ユニークキー)** とは似ていますが、前者は NULL を許容しないのに対し、後者が許容するという点が異なります。**主キー**は、その名の通り、テーブル内のデータを一意に特定するために用いられるもので、テーブル内に複数設定することはできません。ただし、複数の列から構成される**複合キー**は可能です (複合キーを指定するには、後述するオプションとして指定する必要があります)。主キーは必ずしも必須ではありませんが、リレーショナルデータベースの世界では外部テーブルとのリレーションシップを設定するという意味でも、原則として主キーを設定することを強くお勧めします。

主キーと結合

●主キーがあるテーブル

著者情報テーブル	
isbn	author
4-0010-0000-0	山田愛子
4-7981-0959-2	佐藤一郎
4-0010-0000-0	田中太郎
4-8833-0000-1	守口靖男
4-0010-0000-4	山田祥寛

書籍テーブル			
isbn	title	price	...
4-0010-0000-0	ハムスターの観察	900	...
4-7981-0959-2	PEAR 入門	3000	...
4-7980-0945-8	PHP5 サンプル集	3000	...
4-8833-0000-1	SQL リファレンス	2500	...
4-0010-0000-4	フェレットの観察	1000	...

常に 1 つのレコードと結合できる

●主キーがないテーブル

著者情報テーブル	
isbn	author
4-0010-0000-0	山田愛子
4-7981-0959-2	佐藤一郎
4-0010-0000-0	田中太郎
4-8833-0000-1	守口靖男
4-0010-0000-4	山田祥寛

書籍テーブル			
isbn	title	price	...
4-0010-0000-0	ハムスターの観察	900	...
4-0010-0000-0	ハムスターの観察	900	...
4-7980-0945-8	PHP5 サンプル集	3000	...
4-8833-0000-1	SQL リファレンス	2500	...
4-0010-0000-4	フェレットの観察	1000	...

一意なキーがないので、どちらと結合したらよいのか分からない

AUTO_INCREMENT はレコードを追加するごとに自動的に連番を振るためのしくみで、一般的には ID 値を生成するために使用します。

DEFAULT はその名の通り、列に対してデフォルト値を指定するためのフラグです。デフォルト値を指定した列で、INSERT 時に明示的に値が渡されなかった場合、自動的にデフォルト値が該当行にセットされます。

REFERENCES は、外部参照制約を指定するためのフラグです。P.11 でも示したように、リレーショナルデータベースはテーブル間で主キーと外部キーとが対応する値を持つことでリレーションシップ（結合関係）を表現しています。外部参照制約は、この対応関係が矛盾なく保たれていることを確認するためのフラグです。

NOTE

ただし、Oracle や PostgreSQL では AUTO_INCREMENT には対応していません。PostgreSQL では擬似データ型である SERIAL を利用することで、内部的に連番を管理するためのシーケンスが作成され、AUTO_INCREMENT と同様の機能を実現できます（ユーザがシーケンスを意識する必要はありません）。一方、Oracle では明示的にシーケンスを利用して、シーケンスから連番を取得してする必要があります。シーケンスを作成するには、CREATE SEQUENCE 命令を使用します。quest_seq はシーケンス名とします。

```
CREATE SEQUENCE quest_seq;
```

シーケンスを利用したデータの登録については、巻末解答集の 3-2 の問 1 の説明を参考にしてください。

また、SQLite では AUTOINCREMENT キーワードを、SQL Server では IDENTITY キーワードを代わりに使用する必要があります。

次に、オプションについて説明します。オプションはテーブルに含まれる列のキーやインデックス（後述）などの制約条件を指定するものです。列定義の後にカンマ (,) 区切りで記述します。以下には主要なオプションを紹介しておきます。

●主要なオプション

オプション	内容
PRIMARY KEY (列名 ...)	主キーを作成
INDEX インデックス名 (列名, ...)	インデックスを作成 (NULL 値は不可)
UNIQUE インデックス名 (列名, ...)	重複した値を許さない (NULL 値は不可)
FOREIGN KEY (列名) REFERENCES テーブル名 (列名, ...)	外部参照制約

なお、既存のテーブルを破棄するには、以下のように DROP TABLE 命令を使用します。DELETE 命令の項でも述べましたが、DROP TABLE 命令のようにデータを削除する命令を使用する場合には、そのテーブルが本当に不要であるのか十分に検討してから行うようにしてください。いったん削除されてしまったテーブルは元に戻すことはできません。

```
> DROP TABLE quest;
Query OK, 0 rows affected (0.03 sec)
```

SQL 命令を記述してみよう

CREATE TABLE 命令を用いたテーブル生成の基本的な流れを辿ってみることにし

ましょう。

STEP 1 なにをしたいの？

これまでと同様、まずはデータベースに対して「なにをしたいのか」を伝える必要があります。ここでは、テーブルを作成したいので、「CREATE TABLE」を指定します。

```
CREATE TABLE
;
```

STEP 2 テーブル名は？

「テーブルを作成 (CREATE TABLE)」する場合、次にテーブル名を指定する必要があります。また、詳細の定義を記述するためのカッコを用意しておきます。

```
CREATE TABLE
  quest
(
)
;
```

STEP 3 含まれる列は？

テーブルに含まれる列定義をカンマ (,) 区切りで指定します。1つの列定義には「列名 データ型 列フラグ ...」の形式で列に関する情報を記述します。列フラグは互いに矛盾しない限り、複数併記できます。

```
CREATE TABLE
  quest
(
  id INT AUTO_INCREMENT,
  name VARCHAR(100) NOT NULL,
  name_kana VARCHAR(255) NOT NULL,
  sex VARCHAR(5) NOT NULL,
  prefecture VARCHAR(10) NOT NULL,
  age INT DEFAULT 0,
  answer1 INT NULL,
  answer2 TEXT NULL,
  answered DATETIME NOT NULL
)
;
```

STEP 4 制約条件は？

必要に応じて、列定義の後方に制約条件 (オプション) を設定することができます。ここでは、PRIMARY KEY (主キー) の設定だけしておきますが、複数のオプションを追加したい場合には、カンマ区切りで適宜、追加できます。

```
CREATE TABLE
  quest
(
  id INT AUTO_INCREMENT,
  name VARCHAR(100) NOT NULL,
  name_kana VARCHAR(255) NOT NULL,
  sex VARCHAR(5) NOT NULL,
  prefecture VARCHAR(10) NOT NULL,
  age INT DEFAULT 0,
  answer1 INT NULL,
  answer2 TEXT NULL,
  answered DATETIME NOT NULL,
  PRIMARY KEY (id)
)
;
```

完成!

これで1つのSQL命令が完成です。

NOTE

本項の例題では、主キーをオプションとして指定していますが、列フラグで置き換えることも可能です。列フラグを使用した場合には、以下のように記述します。

```
CREATE TABLE
  quest
(
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  name_kana VARCHAR(255) NOT NULL,
  sex VARCHAR(5) NOT NULL,
  prefecture VARCHAR(10) NOT NULL,
  age INT DEFAULT 0,
  answer1 INT NULL,
  answer2 TEXT NULL,
  answered DATETIME NOT NULL
)
;
```


マスタ・プラクティス



問 1

著者情報テーブル (author) を新規に作成してみましょう。以下の空欄を埋めて、SQL命令を完成させてください。

```
CREATE TABLE
  ①
(
  author_id CHAR(5) ②,
  name VARCHAR(30),
  name_kana VARCHAR(100),
  birth ③
);
```

問 2

注文明細テーブル (order_desc) を新規に作成してみましょう。以下の空欄を埋めて、SQL命令を完成させてください。

```
①
order_desc
(
  po_id INT NOT NULL,
  p_id CHAR(10) NOT NULL,
  ②,
  ③ (po_id, p_id)
);
```

問 3

月間売り上げテーブル (sales) を新規に生成してみましょう。

問 4

書籍情報テーブル (books) を新規に作成してみましょう。

問 5

以下は、貸し出し記録テーブル (rental) を新規に作成するためのSQL命令ですが、誤りが2点あります。誤りを指摘してください。

```
CREATE TABLE INTO
rental
(
  id INT AUTO_INCREMENT, PRIMARY KEY,
  user_id CHAR(7),
  isbn CHAR(13),
  rental_date DATE,
  returned SMALLINT DEFAULT 0
);
```


4-2 新規にインデックスを作成したい CREATE INDEX 命令

MySQL PostgreSQL SQLite SQL Server Oracle



Case インデックスはテーブルに対する検索効率を向上するための重要な要素です。インデックス作成の構文は単純なものですので、構文そのものを理解するよりもむしろ、どのようなルールでインデックスを作成するかの方が重要です。そうした視点で、本項は学習してみましょう。

Q 書籍情報テーブル (books) に対して、出版社、刊行日の順で複合インデックスを作成してみましょう。

A インデックスを作成するには、CREATE INDEX 命令を使用します。

```
> CREATE INDEX
>   pub_date
> ON
>   books
>   (
>     publish,
>     publish_date
>   )
> ;
Query OK, 0 rows affected (0.19 sec)
Records: 0 Duplicates: 0 Warnings: 0
```

isbn	...	publish	...	publish_date
4-0010-0000-0	...	山田出版	...	2010-11-01
4-7981-0959-2	...	翔泳社	...	2012-09-08
4-7980-0945-8	...	秀和システム	...	2012-11-01
4-8833-0000-1	...	日経BP	...	2013-02-15
4-0010-0000-4	...	山田出版	...	2012-10-26

出版社、刊行日について索引を作成

publish	publish_date
秀和システム	2012-11-01
翔泳社	2012-09-08
日経BP	2013-02-15
山田出版	2010-11-01
山田出版	2012-10-26

構文を理解しよう

構文を理解する前に、まずはインデックスについて簡単に説明しておきます。

インデックス (Index: 索引) とは、その名の通り、書籍の索引のようなものです。書籍に含まれる膨大な情報の中から特定の記事やトピックを探したいという場合、書籍の先頭から順番に探していくのは、言うまでもなく手間がかかる作業です。そして、これはいかに高速な作業が得意なコンピュータであっても同様でしょう。

そこで、あらかじめ特定の列 (セット) についてのインデックスを作成しておくことで、その列をキーに検索する場合の処理速度を向上できるというわけです。適切なインデックスを定義することは、検索パフォーマンスを左右する重要なポイントです。

インデックスの役割

isbn	title	...	publish
4-0010-0000-0	ハムスターの観察	...	山田出版
4-7981-0959-2	PEAR 入門	...	翔泳社
4-7980-0945-8	PHP5 サンプル集	...	秀和システム
4-8833-0000-1	SQL リファレンス	...	日経BP
4-0010-0000-4	フェレットの観察	...	山田出版
4-7981-0722-0	XML 辞典	...	翔泳社
4-7980-0522-3	JSP リファレンス	...	秀和システム
4-8833-0000-2	SQL プチブック	...	日経BP
4-8833-0000-3	XML リファレンス	...	日経BP
4-0010-0000-9	SQL 入門	...	山田出版
4-0010-0000-1	PHP ドリル	...	山田出版

インデックス

title
JSP リファレンス
PEAR 入門
PHP5 サンプル集
PHP ドリル
SQL 入門
SQL プチブック
SQL リファレンス
XML 辞典
XML リファレンス
ハムスターの観察
フェレットの観察

あらかじめ昇順に並んでいるので、探しやすい

全テーブルを操作するのは時間がかかる!

インデックスを定義する CREATE INDEX 命令の構文は、以下の通りです。

構文 CREATE INDEX 命令

CREATE [UNIQUE] INDEX インデックス名 ON テーブル名 (列名, ...);

索引を作成しなさい 名前は? どのテーブル上に? 索引のキー列は?

UNIQUE キーワードを指定した場合、インデックスに含まれる (複合) 列の内容は重複を許しません。

このようにインデックスを作成するのは簡単ですが、だからといって、すべてのテーブル、すべての列にインデックスを作成すればよいかというと、これは「断じて否」です。というのも、インデックスは検索効率を改善する反面で、以下のようなデメリットがあるからです。

インデックス作成のデメリット

- ・インデックスもデータの集合なので、ディスク領域を圧迫する
- ・索引の更新が発生するため、挿入、更新、削除のパフォーマンスが低下する
- ・索引のヒット件数が多い場合など、条件によっては、インデックスを走査することで却って非効率になる場合がある

インデックスは無条件になんでも作成すれば良いというものではありません。以下では、インデックスを作成する場合のガイドラインを示しておくことにします。

インデックス作成の目安

- ・テーブルに含まれるデータ件数が多い
- ・検索頻度が多い
- ・該当列が検索キーとなる、または外部キーである
- ・該当列によって、データ件数がテーブル全体の1割未満にまで絞り込める

これらの規則は、必ずしも一概に言えるものではありませんが、インデックスを定義する場合の1つの目安にしてください（実際のパフォーマンスは環境やデータ件数、アクセス状況などを見つつ、レスポンス時間を計測しながら、適時、チューニングしていくべきものです）。

既存のインデックスは、**DROP INDEX** 命令によって削除できます。

```
> DROP INDEX pub_date ON books;
Query OK, 0 rows affected (0.17 sec)
Records: 0 Duplicates: 0 Warnings: 0
```

ただし、DROP INDEX 命令の記法はデータベースによって異なる場合があります（以下参照）。

```
DROP INDEX pub_date; (Oracle, PostgreSQL, SQLite)
DROP INDEX books.pub_date; (SQL Server)
```

NOTE

もっとも、インデックスを設定したからと言って、必ずしも検索時にインデックスが使用されるとは限りません。たとえば、以下のようなケースでは、インデックスは使用されませんので、注意が必要です。

●インデックスが使用されない例

- ① IS NULL、IS NOT NULL、<>、LIKE 演算子などを使用した場合（ただし、LIKE 演算子による前方一致検索では使用される）
- ② インデックス列に対して演算や関数を適用している（Ex. price * 1.05 < 3000）
- ③ 複合インデックスを設定しており、かつ、インデックスの先頭列が WHERE 句に含まれていない場合

- ①、③のケースではインデックスのキー列そのものを今一度検討すべきです。また、①の LIKE 演算子については P.65 でも紹介したように、可能な限り、完全（前方）一致でニーズを満たさないか、要件から見直してみることも重要でしょう。
- ②については、演算を右辺で行うことで改善できます。たとえば、「price * 1.05 < 3000」ならば「price < 3000 / 1.05」のように計算式を右辺に集めることでインデックスが使用されるようになります。



SQL 命令を記述してみよう

CREATE INDEX 命令を用いたインデックス生成の基本的な流れを辿ってみることにしましょう。

STEP 1 なにをしたいの？

これまでと同様、まずはデータベースに対して「なにをしたいのか」を伝える必要があります。ここでは、インデックスを作成したいので、「CREATE INDEX」を指定します。

```
CREATE INDEX
;
```

STEP 2 インデックス名は？

「インデックスを作成 (CREATE INDEX)」する場合、次にインデックス名を指定する必要があります。


```
CREATE INDEX
  pub_date
;
```

STEP 3 対象のテーブルは？

インデックスを作成するもとなる対象のテーブルを指定します。テーブル指定は、ON 句で行います。

```
CREATE INDEX
  pub_date
ON
  books
;
```

STEP 4 対象列は？

最後に、インデックスの対象となる列（リスト）を指定します。**複合インデックス**（複数の列から構成されるインデックス）を作成する場合には、列名をカンマ（,）区切りで記述します。

```
CREATE INDEX
  pub_date
ON
  books
(
  publish,
  publish_date
)
;
```

完成！

これで1つのSQL命令が完成です。

NOTE

作成したインデックスの内容を確認するには、たとえば MySQL の場合、**SHOW** 命令を使用します（ただし、確認の方法はデータベースによって異なります）。

```
> SHOW INDEX FROM
>   books
> ;
```

Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Null	Index_type	Comment
books	0	PRIMARY	1	isbn	A	0	NULL	NULL		BTREE	
books	1	pub_date	1	publish	A	0	NULL	NULL	YES	BTREE	
books	1	pub_date	2	publish_date	A	0	NULL	NULL	YES	BTREE	

3 rows in set (0.00 sec)

マスタ・プラクティス



問 1

注 本項の SQL 命令を実行するには、あらかじめ元となるテーブルを作成しておく必要があります。テーブル作成の SQL 命令については、配布サンプルの（データベース名）.sql を参考にしてください。

貸し出し記録テーブル（rental）上に貸出日をキーにしたインデックスを作成してみましょう。以下の空欄を埋めて、SQL命令を完成させてください。

```

①
ind_rental
②
rental
(
  ③
)
;
  
```

問 2

注文書テーブル（order_main）上に発注日、納品日をキーにした複合インデックスを作成してみましょう。以下の空欄を埋めて、SQL命令を完成させてください。

```

①
ind_order
ON
  ②
(
  ③
)
;
  
```

問 3

商品テーブル（product）上に商品名をキーとしたインデックスを作成してみましょう。インデックス名はind_productとします。

問 4

以下は、ユーザーテーブル（usr）に対して住所（都道府県、市町村、その他）で複合インデックスを作成するためのSQL命令ですが、誤りが2点あります。誤りを指摘してください。

```

CREATE INDEX
  ind_usr
IN
  usr
(
  prefecture
  city
  o_address
)
;
  
```


4-3 既存テーブルに列や制約条件を追加したい ALTER TABLE 命令

MySQL PostgreSQL SQLite SQL Server Oracle

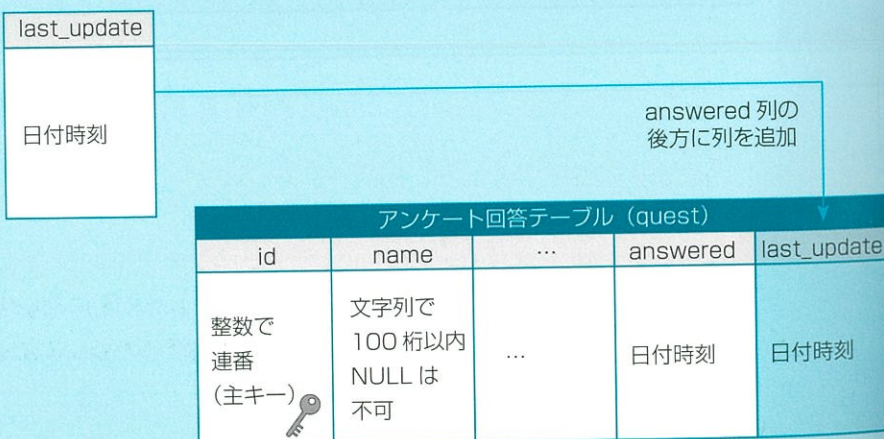


いったん作成したテーブルを運用しているうちに、新たに列を追加する必要が出てきた、主キーやインデックスなどの制約条件を追加する必要があるが発生した、という局面は多くあります。そんなときにもテーブルの一部の列や制約条件を後から追加することができます。

Q アンケート回答テーブル (quest) の末尾に、新規の列として「DATETIME 型の last_update (最終更新日)」を追加してみましょう。

A テーブルの定義内容を追加するには、ALTER TABLE 命令を使用します。

```
> ALTER TABLE
>   quest
> ADD
>   last_update DATETIME
> AFTER
>   answered
> ;
Query OK, 0 rows affected (0.19 sec)
Records: 0  Duplicates: 0  Warnings: 0
```



4-3 既存テーブルに列や制約条件を追加したい

構文を理解しよう

テーブルの定義内容を変更する ALTER TABLE 命令の一般的な構文は以下の通りです。列を追加するには、ADD 句を使用します。

構文 ALTER TABLE 命令 (1)

ALTER TABLE テーブル名 ADD 追加内容 ;

テーブル定義を変更しなさい 名前は何? 変更内容は?

※ ただし、Oracle では追加内容をカッコでくくる必要があります (以降も同様)

日本語的に解くならば、「<テーブル名>の定義に追加内容を追加しなさい」というわけです。ここでは、テーブル名は quest、追加内容として「last_update DATETIME」という列定義が指定されていますので、「quest テーブルに対して last_update 列を追加しなさい」という意味になります。複数の列を追加する場合には、同じように ADD 句を列記します。

AFTER 句は、列を追加する場合にのみ利用できる句で、列をテーブル内のどこに追加するかを指定できます。「AFTER 列名」の代わりに、「FIRST」を指定した場合には、列の先頭に新規列が追加されます (ただし、挿入箇所の指定は MySQL 固有の記法です。他のデータベースでは利用できません)。

なお、ALTER TABLE 命令では列だけではなく、制約条件も追加できます。たとえば、主キーを追加するならば、以下のように記述します (制約条件の記述方法については、CREATE TABLE 命令を参照してください)。もちろん、既にテーブル上に主キーが設定されている場合には、以下の命令は失敗します。

```
ALTER TABLE
quest
ADD
PRIMARY KEY (id)
;
```

列や制約条件を削除するには、ALTER TABLE 命令で DROP 句を使用します。たとえば、以下は quest テーブルから年齢 (age) 列を削除するための SQL 命令です。


```
ALTER TABLE
  quest
DROP
  age
;
```

※SQL Server では DROP 句の代わりに、DROP COLUMN 句を使用します。SQLite では、列の削除はできません。

主キーを削除したい場合には、以下のようにしてください。ただし、主キーの削除はSQL ServerやSQLiteなど、データベースによっては対応していないものもありますので、注意してください。

```
ALTER TABLE
  author
DROP
  PRIMARY KEY
;
```

DELETE や DROP 命令の項でも述べましたが、列を削除する場合にはあらかじめ該当列が本当に不要なのかをよく検討してください。列を削除した場合、当然、列の中身はすべて削除され、元に戻すことはできません。

SQL 命令を記述してみよう

ALTER TABLE 命令を用いたテーブル定義追加の基本的な流れを辿ってみることにしましょう。

STEP 1 なにをしたいの？

例によって、まずはデータベースに対して「なにをしたいのか」を伝えてみましょう。ここでは、テーブルを変更するための「ALTER TABLE」を指定します。

```
ALTER TABLE
;
```

STEP 2 変更したいテーブルは？

「テーブルを定義変更 (ALTER TABLE)」する場合、次に変更対象のテーブル名を指定する必要があります。

```
ALTER TABLE
  quest
;
```

STEP 3 どう変更したいの？

ALTER TABLE 命令は、テーブル定義の追加、更新、削除など変更全般を行うための命令です。ここでは、テーブルに列定義を追加したいので、ADD キーワードを追加します。

```
ALTER TABLE
  quest
ADD
;
```

STEP 4 追加列は？

次に、追加列を指定します。列定義の記法は、CREATE TABLE 命令で使用したものと同様、「列名 データ型 列フラグ ...」の形式です。

```
ALTER TABLE
  quest
ADD
  last_update DATETIME
;
```

STEP 5 追加先は？

最後に、列を追加する先を指定します。「AFTER 列名」で指定された列の直後に新規列が追加されます。先頭列として追加したい場合には、「FIRST」を指定してください。

```
ALTER TABLE
  quest
ADD
  last_update DATETIME
AFTER
  answered
;
```

完成！

これで1つのSQL命令が完成です。

マスタ・プラクティス

問 1

書籍情報テーブル (books) に対して、pages列 (INT型、デフォルト値は0)、rating列 (CHAR(1)型、デフォルトは'B') を新たに追加してみましょう。追加先は、テーブル定義の一番最後とします。空欄を埋めて、SQL命令を完成させてください。

```

①
books
ADD
  ②,
ADD
  rating CHAR(1) DEFAULT 'B' NOT NULL
AFTER
  ③
;

```

問 2

月間売り上げテーブル (sales) を主キーのない状態でまず新規作成してみましょう。これに対して、後から主キーを追加します。以下の空欄を埋めて、SQL命令を完成させてください。

```

①
sales
(
  ②,
  s_date DATE NOT NULL,
  s_value INT DEFAULT 0
)
;
③
sales
ADD
  ④
;

```

問 3

書籍情報テーブル (books) のcategory_idの後方にsales列 (INT型) を追加してみましょう。

問 4

ユーザテーブル (usr) のf_name_kana列の後方に以下の2列を追加してみましょう。

- ・sex列 (VARCHAR(5)型、デフォルト値は男)
- ・job列 (VARCHAR(30)型)

問 5

以下は、社員テーブル (employee) のdepart_id列の後方に「email列 (VARCHAR(255)型)」を追加するSQL命令ですが、2点誤りがあります。誤りを指摘してください。

```

ALTER TABLE
  employee
ADD TO
  email VARCHAR(255) NOT NULL
BEFORE
  depart_id
;

```


4-4 既存テーブル上の列や制約条件を変更したい ALTER TABLE 命令

MySQL PostgreSQL SQLite SQL Server Oracle



Case 前項では、既存のテーブル定義に対して新規の列や制約条件を追加する方法について紹介しました。引き続き SQL を利用して、既存列や制約条件を変更する方法について学びます。

Q ユーザテーブル (usr) 上の o_address 列定義を VARCHAR(255) 型、NULL を許可の条件で変更してみましょう。

A 列定義を変更するには、ALTER TABLE 命令で MODIFY 句を使用します。

```
> ALTER TABLE
>   usr
> MODIFY
>   o_address VARCHAR(255) NULL
> ;
Query OK, 0 rows affected (0.19 sec)
Records: 0 Duplicates: 0 Warnings: 0
```

ユーザテーブル (usr)				
user_id	...	o_address	...	email
文字列で 7 桁 (主キー)		文字列で 100 桁 以内		文字列で 255 桁以内

指定列の定義を
変更

ユーザテーブル (usr)				
user_id	...	o_address	...	email
文字列で 7 桁 (主キー)		文字列で 255 桁以内		文字列で 255 桁以内

4-4 既存テーブル上の列や制約条件を変更したい

構文を理解しよう

MODIFY 句を使用した ALTER TABLE 命令の一般的な構文は以下の通りです。

構文 ALTER TABLE 命令 (2)

ALTER TABLE テーブル名 MODIFY 変更内容 ;

テーブル定義を変更しなさい 名前は何? 変更内容は?

※SQL Server、PostgreSQL では MODIFY 句の代わりに ALTER COLUMN 句を使用します。SQLite では、列定義の変更はできません。

例によって、日本語的に読み解くならば、「<テーブル名>の定義を<変更内容>で修正しなさい」ということになります。ここでは、テーブル名として usr、変更内容として「o_address VARCHAR(255) NULL」という列定義が指定されていますので、「usr テーブル上の o_address 列を「VARCHAR(255)、NULL 許可」で再定義しなさい」という意味になります。複数の列を修正する場合には、同じように MODIFY 句を列記します。

なお、テーブル名を変更したい場合には、以下のように RENAME AS 句 (MySQL のみ。Oracle、PostgreSQL、SQLite では RENAME TO 句) を使用します。old_name が古いテーブル名、new_name が新しいテーブル名とします。

```
ALTER TABLE
old_name
RENAME AS
new_name
;
```

SQL 命令を記述してみよう

ALTER TABLE...MODIFY 句を用いたテーブル定義変更の基本的な流れを辿ってみましょう。

STEP 1 なにをしたいの?

例によって、まずはデータベースに対して「なにをしたいのか」を伝えてみましょう。ここでは、前項同様、テーブルを変更するための「ALTER TABLE」を指定します。

```
ALTER TABLE
;
```


STEP 2 変更したいテーブルは？

「テーブルを定義変更 (ALTER TABLE)」する場合、次に変更対象のテーブル名を指定する必要があります。

```
ALTER TABLE
  usr
;
```

STEP 3 どう変更したいの？

ALTER TABLE 命令は、テーブル定義の追加、更新、削除など変更全般を行うための命令です。ここでは、テーブルの列定義を変更したいので、MODIFY 句を追加します。

```
ALTER TABLE
  usr
MODIFY
;
```

STEP 4 変更列は？

最後に、変更列を指定します。列定義の記法は、CREATE TABLE 命令で使ったものと同様、「列名 データ型 列フラグ ...」の形式です。

```
ALTER TABLE
  usr
MODIFY
  o_address VARCHAR(255) NULL
;
```

完成！

これで1つのSQL命令が完成です。

マスタ・プラクティス



問 1

書籍テーブル (books) 上、publish列の定義を「VARCHAR(100) NULLを許可」で変更してみましょう。以下の空欄を埋めて、SQL命令を完成させてください。

```

①
books
MODIFY
  ②
;
```

問 2

社員テーブル (employee) 上、社員の氏・名定義をいずれも「VARCHAR(50) NULLを許可しない」に変更してみましょう。以下の空欄を埋めて、SQL命令を完成させてください。

```

ALTER TABLE
  ①
  ②
  l_name VARCHAR(50) NOT NULL,
  ③
;
```

問 3

著者情報テーブル (author) 上、著者名の定義を「VARCHAR(100) デフォルト値は空文字列」に変更してみましょう。

```

ALTER TABLE
  author
MODIFY
  author_name VARCHAR(100) DEFAULT ''
;
```


問 4

以下は、アクセス記録テーブル (access_log) に対して、既存のreferer列の定義を「データ型VARCHAR(200)、NULL値を許可」で置き換えるためのSQL命令ですが、2点誤りがあります。誤りを指摘してください。

```
ALTER TABLE  
  access_log  
MODIFY WITH  
  referer, VARCHAR(200), NULL  
;
```

解答集

マスタ・プラクティス